

Hardwarové platformy

Na jakém stroji náš program běží – architektury, AVR a ARM
Programování – 3. ročník (Mechatronika), téma 01

Jaroslav Bušek

Střední průmyslová škola na Proseku

Co dnes zvládneme

- Vysvětlím, co je **hardwarová platforma** a proč se stejný program nepřeloží pro každý procesor.
- Popíšu cestu od zdrojového kódu k programu v paměti mikrokontroleru.
- Porovnáám **AVR a ARM** a u zadané úlohy zdůvodním, kterou platformu bych vybral.

Učivo dle ŠVP: U1 – možnosti tvorby programů pro různé hardwarové platformy,
U2 – vybrané platformy (ARM, AVR), jejich přednosti a omezení.

Kolik mikrokontrolerů jste dnes už použili?

Chytrá
technologie
kolem nás

Kde všude je **mikrokontroler**?

Malé čipy,
velké možnosti
v našem světě!



Pračka



Řídí celý
prací cyklus



Myš



Zpracovává
pohyb a kliknutí



Klíček od auta



Řídí komunikaci
s autem



Elektrokolo



Řídí výkon motoru,
sleduje rychlost
a další funkce



Nabíječka



Řídí nabíjení,
a chrání zařízení



Ovladač televize



Zpracovává stisky
tlačítek a vysílá signál



Mikrokontrolery jsou všude kolem nás – v běžných zařízeních, která denně používáme!

Proč se tím zabýváme

Ročně se vyrobí přes **25 miliard** mikrokontrolerů – víc než procesorů do PC a mobilů dohromady.

Co mají všechny společné

Jedna úloha, žádný operační systém, cena v jednotkách korun, spotřeba v miliampérech, a musí fungovat roky bez restartu.

Náš rok: naučit se pro takový čip napsat program tak, aby fungoval spolehlivě.

Část 1 (U1): co se dozvíte

- 1 Co je hardwarová platforma a co všechno určuje.
- 2 Proč procesor nerozumí jazyku C – a co s tím dělá překladač.
- 3 Jak z naší skici vznikne program ve Flash paměti čipu.
- 4 Na jakých úrovních se dá pro čip programovat a co která stojí.
- 5 Jak jsou v ATmega328P rozdělené paměti a jak se do nich program dostane.

Nejde o detaily. Cílem je orientace: vědět, co se kde děje a jaká slova znamenají co. Podrobnosti přijdou u konkrétních periférií.

Co je hardwarová platforma

Definice

Hardwarová platforma je kombinace procesorového jádra, jeho periferií a desky, na které to všechno je. Pro každou platformu vzniká **jiný** výsledný program – i když zdrojový kód je stejný.

Vrstva	Co určuje	U Arduina UNO
Architektura jádra	instrukční sadu, šířku slova, registry	AVR, 8 bitů
Konkrétní čip	kolik paměti a jaké periferie	ATmega328P
Deska	napájení, konektory, převodník USB	Arduino UNO R3

Když přejdu na jinou platformu

Co zůstane

- algoritmus a návrh programu
- jazyk C/C++
- struktura `setup()` / `loop()`
- způsob myšlení o časování a stavech

Co se změní

- překladač (`avr-gcc` → `arm-gcc`)
- názvy registrů a čísla pinů
- knihovny pro periferie
- napájecí napětí (`5 V` → `3,3 V`)

Proto se to učí takhle: letos na jedné platformě do hloubky. Přechod na druhou pak není nové programování, jen nové jméno pro tu samou věc.

Stejný řádek v C, jiné instrukce

$y = y + 1;$ kde y je bajt v paměti

AVR (ATmega328P, 8 bitů)

```
lds  r24, 0x0100    ; nacti y
subi r24, 0xFF      ; pricti 1
sts  0x0100, r24     ; uloz y
```

ARM Cortex-M (32 bitů)

```
ldr  r3, [pc, #16] ; adresa y
ldrb r2, [r3, #0]  ; nacti y
adds r2, r2, #1    ; pricti 1
strb r2, [r3, #0]  ; uloz y
```

Jiné názvy instrukcí, jiný počet registrů, jiná šířka slova. Procesor nerozumí jazyku C – rozumí **jen své instrukční sadě**. Překladač je ten, kdo náš kód do ní přeloží, a pro každou architekturu je to **jiný překladač**. (Výpisy jsou zjednodušené.)

Křížový překlad – překládám na PC, běží to na čipu

Na PC nemáme AVR procesor, a přesto pro něj umíme vyrobit program. Tomu se říká **křížový překlad** (cross-compile).

Cíl	Sada nástrojů
PC (Windows)	gcc, cl.exe
AVR	avr-gcc, avrdude
ARM Cortex-M	arm-none-eabi-gcc

Arduino IDE tyto nástroje obsahuje a volá je za nás. Proto se instalace „jednoho programu“ postará o celý řetězec.

Slovníček: sada nástrojů pro jednu platformu se souhrnně nazývá **toolchain**.

Kvíz – proč nejde `program.exe` nahrát do Arduina?

- A) Jde to, jen se musí přejmenovat na `.hex`.
- B) Nejde, protože `.exe` je moc velký soubor.
- C) Nejde, protože obsahuje instrukce x86, volání operačního systému a počítá s gigabajty RAM.
- D) Nejde, protože Arduino umí jen jazyk C, ne C++.

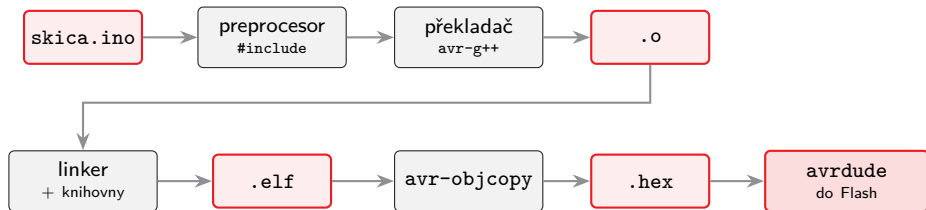
Kvíz – proč nejde `program.exe` nahrát do Arduina?

- A) Jde to, jen se musí přejmenovat na `.hex`.
- B) Nejde, protože `.exe` je moc velký soubor.
- C) **Nejde, protože obsahuje instrukce x86, volání operačního systému a počítá s gigabajty RAM.**
- D) Nejde, protože Arduino umí jen jazyk C, ne C++.

C)

Nic z toho na ATmega328P není: jiná instrukční sada, žádný operační systém, 2 kB RAM. Výsledný program je proto vždycky svázaný s platformou, pro kterou byl přeložený.

Cesta od naší skici do paměti čipu



V Arduino IDE se tohle všechno schová za jedno tlačítko. Když se objeví chyba, je ale dobré vědět, **ve kterém kroku** vznikla: chyba v syntaxi je z překladače, „undefined reference“ z linkeru a „nelze otevřít port“ z nahrávání.

Co Arduino IDE přidá k našemu kódu

```
// tohle do skici nepiseme,  
// pridá se to samo:  
#include <Arduino.h>  
  
void blikni();    // prototypy  
  
int main(void) {  
    init();  
    setup();  
    for (;;) { loop(); }  
}
```

Proto v Arduino funguje volání funkce, která je v souboru **níž** – prototyp za nás doplní IDE.

Důsledek: skica .ino **není čisté C**. Je to C++ s několika doplňky, a běžným gcc se nepřeloží.

V bloku 15 proto budeme kód psát do .h a .cpp – tam už žádná magie není a platí normální pravidla jazyka.

Možnosti tvorby programů (U1)

Pro stejný čip lze psát na několika úrovních. Volba je vždy kompromis:

Úroveň	Jak to vypadá	Rychlost	Pracnost
Asembler	jednotlivé instrukce	nejlepší	velmi vysoká
C/C++ s registry	<code>PORTB = (1<<5);</code>	výborná	vysoká
C/C++ s HAL	<code>HAL_GPIO_WritePin(.., GPIO_PIN_5, GPIO_PIN_SET);</code>	dobrá	střední
Arduino	<code>digitalWrite(13, HIGH);</code>	horší	nízká
MicroPython	<code>pin.value(1)</code>	nejhorší	nejnižší

Letos: Arduino framework – abychom se soustředili na návrh programu. V bloku 03 se podíváme i „pod něj“, na registry: `digitalWrite()` je desítky instrukcí tam, kde by stačila jedna.

Co je HAL – vrstva abstrakce hardwaru

HAL = Hardware Abstraction Layer, vrstva abstrakce hardwaru

Knihovna, obvykle od **výrobce čipu**, která schová práci s registry za funkce s čitelnými názvy. Program pak neříká „*nastav pátý bit v registru PORTB*“, ale „*nastav pin PB5 do jedničky*“.

```
// primo registr (bez HAL)
PORTB |= (1 << 5);

// pres HAL (STM32)
HAL_GPIO_WritePin(GPIOB,
                   GPIO_PIN_5,
                   GPIO_PIN_SET);
```

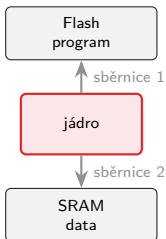
Co za to dostanu: kód se dá přenést na jiný čip téže řady bez prepisování a je čitelnější.

Co to stojí: funkce navíc kontroluje parametry, takže je **pomalejší a delší** než jeden řádek s registrem.

Souvislost: `digitalWrite()` je taky HAL, jen o úroveň výš.

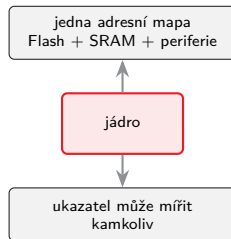
Dvě architektury pamětí

Harvardská – AVR



Oddělené sběrnice i **oddělené adresy**. Adresa 0x0100 ve Flash a v SRAM jsou dvě různá místa.

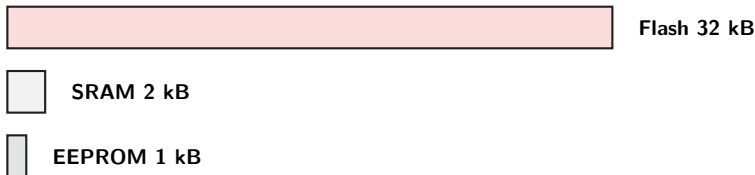
Jednotný prostor – ARM Cortex-M



Jeden adresní prostor. Ukazatel na konstantu ve Flash se používá stejně jako na proměnnou. (Sběrnice uvnitř jádra oddělené být mohou i nemusí – podrobněji v části U2.)

Důsledek pro nás: na AVR se textové konstanty kopírují při startu do SRAM. Proto `Serial.println(F("text"))` – `F()` je nechá ve Flash (blok 09).

Paměti ATmega328P (U7)



To je celá paměť, se kterou letos pracujeme. Program se vejde téměř vždy; co dělá potíže, je **SRAM** – těch 2048 bajtů se spotřebuje překvapivě rychle. Proto se od začátku hlídá, kolik toho který program zabere.

Co je v které paměti

Paměť	Co v ní je	Vlastnosti
Flash	náš program, textové konstanty, 0,5 kB bootloader	po vypnutí zůstane, přepisuje se jen při nahrávání
SRAM	proměnné, pole, zásobník, buffery Serial	po vypnutí se smaže , rychlá, a je jí málo
EEPROM	nastavení, které má přežít vypnutí	zůstane, zapisuje se pomalu a jen ~100 000 krát

Typická otázka u zkoušení: kam patří text "Teplota: "? Do Flash – ale bez F() se při startu zkopíruje i do SRAM, takže zabere obojí.

Jak se program dostane do Flash (U6)

1. Bootloader – běžné nahrávání



2. ICSP – programátorem přímo



Bootloader je **program, který nahrává jiný program**. Po resetu chvíli čeká, jestli po sériové lince nepřijde nový kód; když nepřijde, spustí ten starý.

Bootloader nebo ICSP – kdy co

	Bootloader	ICSP
Potřebuje	jen USB kabel	programátor (USBasp, druhé Arduino)
Cena za to	ubere 0,5 kB Flash, zdrží start	nic
Nový, prázdný čip	nenahraje (bootloader tam ještě není)	nahraje
Kdy se použije	běžná výuka a vývoj	výroba, oživení čipu, potřeba celé Flash

Praktický dopad na nás: celý rok nahráváme přes bootloader a USB. ICSP stačí znát – a vědět, že těch 0,5 kB Flash chybí, když se program „skoro“ vejde.

Shrnutí části 1 (U1)

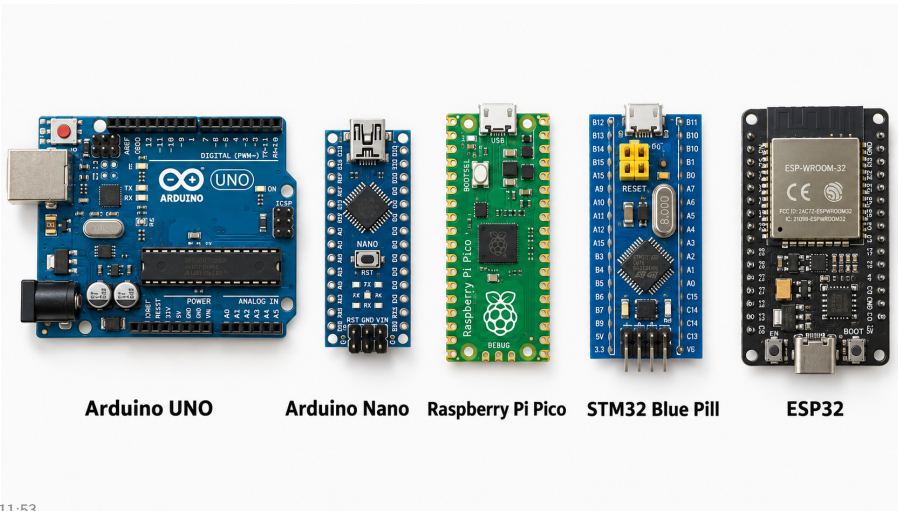
- **Platforma** = jádro + čip + deska. Každé jádro má vlastní instrukční sadu, a tedy vlastní překladač.
- Program se vyrábí **křížovým překladem** na PC: `.ino` → preprocesor → překlad → linker → `.hex` → nahrání do Flash.
- Skica `.ino` není čisté C – IDE k ní přidá `Arduino.h`, prototypy a `main()`.
- Programovat se dá od **assembleru** po **MicroPython**; Arduino framework je kompromis mezi pracností a rychlostí.
- AVR je **harvardská** architektura – Flash a SRAM mají oddělené adresy.
- Flash 32 kB zůstane, **SRAM 2 kB** se maže a je jí málo, EEPROM 1 kB je pomalá.
- Nahrává se přes **bootloader** (jen USB, ubere 0,5 kB) nebo **ICSP** (programátor, uvolní celou Flash).

Část 2 (U2): co se dozvíte

- 1 Co je AVR a co je ARM Cortex-M.
- 2 Jaké má každá z nich přednosti a jaká omezení.
- 3 Čím se liší uvnitř – sada instrukcí, paměť, registry.
- 4 Podle čeho se platforma pro konkrétní úlohu vybírá.
- 5 Co se v téhle oblasti běžně plete – a proč na tom záleží.

Na konci této části byste měli u zadané úlohy umět říct „vzal bych tohle, protože...“ a uvést dva konkrétní důvody.

Desky, o kterých bude řeč



AVR – na čem se letos učíme

8bitová architektura firmy Atmel, na trhu **od roku 1996**. Dnes ji vlastní Microchip a pořád se vyrábí.

Právě ta dlouhá životnost je důvod, proč je kolem ní tolik hotových knihoven, návodů a odpovědí na fóru – a proč se na ní dobře začíná.

ATmega328P v Arduino UNO

- 8 bitů, 16 MHz
- 32 kB Flash, 2 kB SRAM, 1 kB EEPROM
- 20 I/O pinů, ADC 10 bitů
- 3 časovače
- napájení 1,8–5,5 V

AVR – přednosti a omezení

Přednosti

- snese 5 V i kolísavé napájení
- čitelná dokumentace (~300 stran)
- jedna instrukce za takt – časování se dá spočítat
- v patici DIP, dá se vyměnit
- obrovská nabídka knihoven a návodů

Omezení

- 8 bitů – float se počítá softwarově a pomalu
- 2 kB SRAM
- 16 MHz
- žádná Wi-Fi ani USB
- ADC jen 10 bitů

Proč se na tom učí: málo možností znamená, že je vidět, co program dělá. Na 8 bitech a 2 kB se nedá „zamést“ neefektivní řešení pod koberec.

ARM Cortex-M – kam se jde dál

ARM sám čipy nevyrábí – **licencuje jádro** a čipy staví desítky firem (ST, NXP, Nordic, Raspberry Pi). Proto je nabídka tak široká.

Čip	Jádro	Parametry
STM32F103 („Blue Pill“)	Cortex-M3, 72 MHz	64 kB Flash, 20 kB SRAM, ADC 12 b
RP2040 (Pi Pico)	2× Cortex-M0+, 133 MHz	264 kB SRAM, Flash externí
nRF52840	Cortex-M4F, 64 MHz	Bluetooth, 256 kB SRAM

ARM – přednosti a omezení

Přednosti

- 32 bitů, desítky až stovky MHz
- mnohem víc paměti
- bohaté periferie (USB, CAN, Ethernet)
- hardwarový float u M4F
- hluboké režimy spánku

Omezení

- jen 3,3 V – 5 V na pinu čip zničí
- složitější rozběh (hodiny, napájecí domény)
- datasheet má tisíc stran
- čip obvykle nelze vyměnit

Nejčastější zničený čip ve školní laboratoři: ARM, na který někdo přivedl 5 V ze staré desky. Napájecí úroveň se kontroluje **před** zapojením.

Obě jsou RISC – a přesto jiné

RISC (*Reduced Instruction Set Computer*) znamená málo jednoduchých instrukcí, zato rychlých. Opak je CISC – procesory x86 v PC.

	AVR	ARM
Sada instrukcí	RISC, 8 bitů; většina instrukcí za 1 takt	RISC, 32 bitů; sady ARM, Thumb, Thumb-2
Paměť	strikně harvardská	podle jádra: von Neumannova (M0, M0+) nebo modifikovaná harvardská (M3, M4)
Registry	32 registrů po 8 bitech	16 registrů po 32 bitech
Licence	proprietární, Microchip (dříve Atmel)	ARM Ltd. jádro jen navrhuje a licencuje (ST, NXP, Apple)

Cortex-M umí jen Thumb-2, ne původní 32bitovou sadu ARM. Instrukce jsou kratší, takže se program vejde do menší Flash.

Registry – tam, kde se opravdu počítá

Obě architektury jsou typu *load/store*

Aritmetika neumí sáhnout přímo do RAM. Hodnota se musí nejdřív **načíst** do registru, tam se s ní počítá, a výsledek se **uloží** zpět.

AVR

- 32 registrů po 8 bitech, R0–R31
- poslední tři páry slouží jako 16bitové ukazatele X, Y, Z
- registry visí přímo na ALU – operace nad dvěma registry proběhne za jeden takt

ARM Cortex-M

- 16 registrů po 32 bitech, R0–R15
- obecných je jich ale jen 13 (R0–R12)
- R13 = SP (vrchol zásobníku), R14 = LR (návrátová adresa), R15 = PC (další instrukce)

Zpátky k tomu výpisu na začátku hodiny: lds a sts u AVR, ldr a str u ARM – to jsou přesně ty *load* a *store*. Sčítalo se až potom, a jen mezi registry.

Srovnání – co rozhoduje při výběru

	AVR (ATmega328P)	ARM (např. RP2040)
Šířka slova	8 bitů	32 bitů
Frekvence	16 MHz	133 MHz
SRAM	2 kB	264 kB
Napájení	1,8–5,5 V (snese 5 V)	1,8–3,6 V (jen 3,3 V)
float	softwarově, pomalu	rychle (u M4F hardwarově)
Periferie	UART, I2C, SPI, ADC	+ USB, PWM bloky, PIO, DMA
Rozběh čipu	funguje hned	nastavit hodiny a napájení
Dokumentace	~300 stran, čitelná	~600–1000 stran

Pravidlo: nevybírání se „lepší“ čip, ale **nejmenší, který úlohu zvládne**. Rozhoduje spotřeba, napětí periférií, potřebná paměť a cena v sérii.

Kvíz – co byste vybrali a proč?

- 1** Blikající světlo na kolo. Dvě LED, tlačítko, běží z knoflíkové baterie několik měsíců.
- 2** Kamera u vchodu, která posílá snímek do mobilu přes Wi-Fi.
- 3** Regulace otáček motoru s enkodérem, 1000 přepočtů za sekundu s float.

Kvíz – co byste vybrali a proč?

- 1 Blikající světlo na kolo. Dvě LED, tlačítko, běží z knoflíkové baterie několik měsíců.
- 2 Kamera u vchodu, která posílá snímek do mobilu přes Wi-Fi.
- 3 Regulace otáček motoru s enkodérem, 1000 přepočtů za sekundu s float.

	Volba	Proč
1.	AVR (i menší ATtiny)	úloha je triviální, rozhoduje spotřeba ve spánku a cena
2.	ESP32	Wi-Fi musí být na čipu; samotné ARM jádro to neřeší
3.	ARM Cortex-M4F	1000× za sekundu float 8bitový AVR nestihne

Pozor na časté záměny

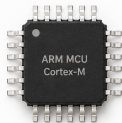
Říká se	Ve skutečnosti
„ESP32 je ARM“	není – má jádro Xtensa (novější řady i RISC-V)
„Arduino je procesor“	Arduino je prostředí a knihovny ; běží na AVR, ARM i ESP32
„Raspberry Pi a Pico je totéž“	Pi je počítač s Linuxem , Pico je mikrokontroler
„procesor“ „mikrokontroler“	= mikrokontroler má jádro, paměti i periferie na jednom čipu

Proč na tom záleží: podle těchto slov se hledá dokumentace a knihovny. Kdo hledá „ARM knihovnu“ pro ESP32, nenajde nic.

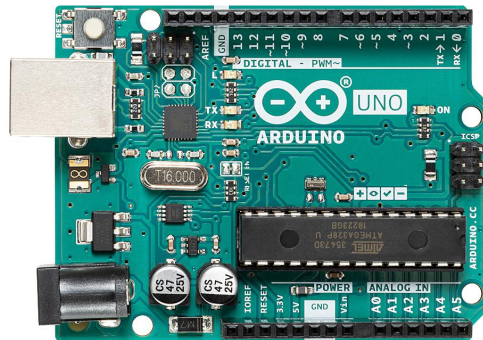
Čipy zblízka (rezerva)



ATmega328P (DIP-28)



ARM čip (LQFP-28)



Na co se dívat: kolik má čip vývodů (tolik nejvýš periférií), v jakém je pouzdře (DIP se dá vyměnit, LQFP už ne) a co všechno je na desce **kolem** čipu – napájení, převodník, krystal. Příště si to celé proměříme multimetrem.

Shrnutí hodiny

- **Platforma** = jádro + čip + deska. Pro každé jádro vzniká jiný výsledný program, protože každé má **jinou instrukční sadu**.
- Program vzniká **křížovým překladem** na PC a nahrává se přes **bootloader** nebo **ICSP**.
- ATmega328P: 32 kB Flash, **2 kB SRAM**, 1 kB EEPROM – a to je všechno.
- AVR: 8 bitů, 5 V, málo paměti, jednoduché a spolehlivé. ARM: 32 bitů, 3,3 V, hodně paměti a periférií, složitější rozběh.
- Vybírá se **nejmenší čip, který úlohu zvládne**.

Příště: Arduino IDE, první nahraná skica, vlastní repozitář a první commit.